

RGI Gripper Integration - Weekly Progress Report

Project: Piper Arm + RGI Rotating Gripper Integration **Date Range:** January 26-30, 2026 **Author:** Robotics UROP Team

Overview

This week focused on integrating the RGI (Rotating Gripper Interface) with the Piper 6-DOF robotic arm to create a complete manipulation system with trajectory recording, playback, and web-based remote control.

Day 1: Initial Investigation & Setup

Objectives

- Investigate Piper arm teach mode capabilities
- Understand RGI gripper communication protocol

Accomplishments

- **Teach Mode Discovery:** Found that teach mode (`ctrl_mode == 2`) requires the physical TEACH button on the controller - no programmatic activation possible
- **SDK Analysis:** Explored `piper_sdk` for arm control methods
- **MIT Mode Testing:** Attempted MIT mode with zero stiffness as alternative (unsuccessful for this use case)
- **Existing Code Review:** Analyzed `recordTrajectory_new_en.py` and `playTrajectory_new_en.py`

Technical Findings

```
# Teach mode detection
ctrl_mode = piper.GetArmStatus().arm_status.ctrl_mode
# ctrl_mode == 1: CAN control mode
# ctrl_mode == 2: Teach mode (requires physical button)
```

Day 2: Created play_and_adjust.py

Objectives

- Combine trajectory playback with gamepad control
- Integrate RGI gripper functionality

Accomplishments

- **Created** `play_and_adjust.py` : Main control script combining:
 - Trajectory playback from CSV
 - Gamepad fine-tuning after trajectory
 - RGI gripper control (open/close/rotate)
- **Integrated Components:**
 - `playTrajectory_new_en.py` (trajectory playback)
 - `rotation_arm_gamepad.py` (RGI gripper control)
- **Changed Playback Mode:** From point-by-point confirmation to full trajectory playback

Files Created

- `rgi_gripper/play_and_adjust.py`
-

Day 3: Bug Fixes & Feature Additions

Objectives

- Fix serial communication issues
- Add essential safety features

Accomplishments

Critical Bug Fix: Serial Port Threading Conflict

Problem: `SerialException: device reports readiness to read but returned no data`

Cause: Main thread calling `rgi.write_reg()` while background thread also accessing serial port

Solution: Added thread-safe methods with lock protection:

```
def close_gripper(self):  
    """Close gripper (thread-safe)"""
```

```

with self.lock:
    if self.ser:
        cmd = self.build_cmd(1, 0x06, 0x0103, 0)
        self.ser.reset_input_buffer()
        self.ser.write(cmd)
        time.sleep(0.02)
        try:
            self.ser.read(100)
        except:
            pass

```

Features Added

1. **Go Home First:** Arm returns to home position before starting any operation
2. **Rotation After Trajectory:** Close gripper + rotate after trajectory completes
3. **Go Home After Gamepad:** Returns home after exiting gamepad mode
4. **Open Gripper Before Home:** Safety feature to prevent collisions

Day 4: Trajectory Management System

Objectives

- Implement multi-trajectory recording
- Add trajectory management features

Accomplishments

Recording System

- **Manual Duration Recording:** Press ENTER to stop (no fixed duration)
- **Named Trajectories:** User-provided names during recording
- **JSON Storage:** Trajectories saved to `saved_positions.json`

Management Features

1. **Record New Trajectory (Option 2)**
2. Goes home first
3. Waits for TEACH mode
4. Records until user presses ENTER
5. **List Saved Trajectories (Option 3)**
6. Shows name, point count, recording date

7. Play Saved Trajectory (Option 4)

8. Select by number or name

9. Full playback with timing

10. Manage Trajectories (Option 5)

11. **Delete (d)**: Remove trajectory permanently

12. **Rename (r)**: Change trajectory name

13. **View (v)**: See details (points, duration, first 5 waypoints)

14. **Replace (x)**: Re-record with same name

Position Actions Menu

After trajectory playback: - Rotate gripper (specify degrees + CW/CCW) - Close gripper - Open gripper - Done (continue)

Day 5: Web Controller & Final Fixes

Objectives

- Create web-based remote control interface
- Fix remaining issues

Accomplishments

Web Controller (`web_controller.py`)

Created Flask-based web interface with:

Features: - Live Orbbec RGB camera feed (MJPEG stream) - Real-time joint position display (WebSocket updates) - Control buttons: Go Home, Open/Close Gripper, Rotate - Trajectory management: List, Play, Delete - Activity log

Technical Stack: - Flask + Flask-SocketIO - OpenCV for camera capture - Threading for non-blocking operations

Access: - Local: `http://localhost:5000` - Network: `http://<IP>:5000`

Bug Fixes

1. **CAN Mode Switching:** Added 3 retry attempts with better error handling
2. **Quit Behavior:** Arm always goes home before shutting down
3. **Teach Mode Timeout:** Reduced from 30s to 15s

4. **Camera Integration:** Fixed Orbbec camera detection at `/dev/video4`

Final File Structure

```
rgi_gripper/
├─ play_and_adjust.py      # Main control script (35KB)
├─ web_controller.py       # Web interface (28KB)
├─ saved_positions.json    # Trajectory storage (436KB)
├─ rotation_arm_gamepad.py # Original gamepad control
├─ gamepad_control.py      # Standalone gamepad
├─ start_web_server.sh     # Quick launch script
├─ README.md              # Documentation
└─ MANUAL_ENGLISH.md      # RGI gripper manual
```

Key Technical Details

RGI Gripper Protocol (Modbus RTU)

Register	Function	Values
0x0100	Initialize	0xA5
0x0103	Gripper Position	0-1000
0x0109	Rotation	Degrees (signed)
0x0104	Gripper Speed	0-100
0x0107	Rotation Speed	0-100

Piper Arm Control

Mode	Value	Description
CAN Control	1	Normal operation
Teach Mode	2	Manual positioning

Usage Summary

Terminal Control

```
cd rgi_gripper && python3 play_and_adjust.py
```

Web Control

```
cd rgi_gripper && python3 web_controller.py  
# Open browser: http://localhost:5000
```

Next Steps (Future Work)

- [] Add trajectory editing (modify waypoints)
 - [] Implement trajectory merging
 - [] Add voice command support
 - [] Improve web UI with trajectory recording
 - [] Add collision detection
-

Generated: January 30, 2026